

Django

Documentação

Como usar o Django com Apache e mod_wsgi

Implementar Django com [Apache](#) e [mod_wsgi](#) é uma forma testada e experimentada de colocar o Django em produção.

O [mod_wsgi](#) é um módulo Apache o qual pode hospedar qualquer aplicação Python [WSGI](#), incluindo Django. Django irá funcionar com qualquer versão do Apache que suporte [mod_wsgi](#).

A [official mod_wsgi documentation](#) é sua fonte para todos os detalhes sobre como usar [mod_wsgi](#). Você provavelmente quer começar com [installation and configuration documentation](#).

Configuração Básica

Um vez que tenha o [mod_wsgi](#) instalado e ativado, edite seu arquivo [http_conf](#) do servidor Apache e adicione o seguinte. Se estiver usando uma versão do Apache anterior a 2.4, substitua **Require all granted** por **Allow from all** e também a linha **Order deny,allow** acima dele.

```
WSGIScriptAlias / /path/to/mysite.com/mysite/wsgi.py
WSGIPythonHome /path/to/venv
WSGIPythonPath /path/to/mysite.com

<Directory /path/to/mysite.com/mysite>
<Files wsgi.py>
Require all granted
</Files>
</Directory>
```

A primeira parte da linha **WSGIScriptAlias** é a URL base que você quer servir sua aplicação (**/** indica a URL raiz), e a segunda é o local do arquivo “WSGI” – veja abaixo – no seu sistema, geralmente dentro do pacote do seu projeto (**mysite** neste exemplo). Isso diz ao Apache para servir qualquer requisição abaixo da URL informada usando a aplicação WSGI definida naquele arquivo.

Se você instalar as dependências Python do seu projeto dentro de um [virtualenv](#), adicione o caminho para o virtualenv usando `WSGIPythonHome`. Veja o guia `mod_wsgi virtualenv` para mais detalhes.

A linha **WSGIPythonPath** assegura que o pacote do seu projeto está disponível no Python path para ser importado; em outras palavras, que **import mysite** funcione.

A parte **<Directory>** somente assegura que o Apache pode acessar seu arquivo **wsgi.py**.

Agora precisamos assegurar que este arquivo **wsgi.py** com objeto de aplicação WSGI exista. Desde de o Django versão 1.4, **startproject** terá criado um para você; se não, você precisará criá-lo. Veja [Documentação geral WSGI](#) para o conteúdo padrão que você deveria colocar neste arquivo, e o que mais você pode adicionar a ele.



Aviso

Se múltiplos sites Django estiverem rodando em um único processo [mod_wsgi](#), todos eles irão usar as definições daquele que executar primeiro. Isso pode ser resolvido alterando:

```
os.environ.setdefault("DJANGO_SETTINGS_MODULE", "{ project_name }.settings")
```

em **wsgi.py**, para:

```
os.environ["DJANGO_SETTINGS_MODULE"] = "{ project_name }.settings"
```

ou por usando o [mode daemon do mod_wsgi](#) e assegurando que cada site está rodando no seu próprio processo daemon.



Arrumando o **UnicodeEncodeError** para uploads de arquivos

Se houver um erro **UnicodeEncodeError** quando fizer o upload de arquivos que contenham caracteres não ASCII, tenha certeza que o Apache está configurado para aceitar arquivos com nomes não ASCII:

<https://docs.djangoproject.com/pt-br/2.2/howto/deployment/wsgi/modwsgi/>

Idioma: **pt-br**

Versão da Documentação: **2.2**

```
export LANG='en_US.UTF-8'
export LC_ALL='en_US.UTF-8'
```

Um lugar comum para colocar essa configuração é em **/etc/apache2/envvars**.

Veja a seção [Files](#) do guia de referência de unicode para detalhes.

Usando mod_wsgi no modo deamon

“Modo Deamon” é o modo recomendado para rodar mod_wsgi (em plataforma não Windows). Para criar o grupo de processos deamon requerido e delegar o Django para rodar dentro, precisa adicionar as diretivas **WSGIDaemonProcess** e **WSGIProcessGroup**. Um outra mudança requerida na configuração acima se usar deamon mode é que não se pode usar o **WSGI PythonPath**; no lugar deve usar a opção **python-path** para **WSGIDaemonProcess**, por exemplo:

```
WSGIDaemonProcess example.com python-home=/path/to/venv python-path=/path/to/mysite.com
WSGIProcessGroup example.com
```

Se quer servir seu projeto em um subdiretório (**https://example.com/mysite** neste exemplo), você pode adicionar **WSGIScriptAlias** a configuração acima:

```
WSGIScriptAlias /mysite /path/to/mysite.com/mysite/wsgi.py process-group=example.com
```

Veja a documentação oficial do mod_wsgi para [details on setting up daemon mode](#)

Servindo arquivos

O Django não serve arquivos, deixa este trabalho para qualquer web server que você escolha.

Recomendamos usar um Web Server separado – isto é, um que não esteja rodando o Django – para servir media. Aqui alguns boas escolhas:

- [Nginx](#)
- Uma versão mais enxuta do [Apache](#)

Se, porém, você não tem opção de servir arquivos de media na mesmo VirtualHost Apache que está o Django, você pode configurar o Apache para servir algumas URLs como media estática, e outros usando a interface mod_wsgi para Django.

Este exemplo define o Django como raiz do site, mas serve **robots.txt**, **favicon.ico**, e qualquer coisa nas URLs **/static/** e **/media** como arquivos estáticos. Todas as outra URLs serão servidas usando o mod_wsgi:

```
Alias /robots.txt /path/to/mysite.com/static/robots.txt
Alias /favicon.ico /path/to/mysite.com/static/favicon.ico

Alias /media/ /path/to/mysite.com/media/
Alias /static/ /path/to/mysite.com/static/

<Directory /path/to/mysite.com/static>
Require all granted
</Directory>

<Directory /path/to/mysite.com/media>
Require all granted
</Directory>

WSGIScriptAlias / /path/to/mysite.com/mysite/wsgi.py

<Directory /path/to/mysite.com/mysite>
<Files wsgi.py>
Require all granted
</Files>
</Directory>
```

Idioma: **pt-br**

Persistindo arquivos do admin

Quando `django.contrib.staticfiles` está em `INSTALLED_APPS`, o servidor de desenvolvimento do Django serve automaticamente os arquivos estáticos da app admin (e de qualquer outra app instalada). Isso não é o caso quando é usado qualquer outro arranjo de servidor. Você é o responsável por configurar o Apache, ou qualquer outro servidor Web que esteja usando, para server arquivos do admin.

Os arquivos do admin ficam em `(django/contrib/admin/static/admin)` da distribuição Django.

Fortemente recomendamos usar `django.contrib.staticfiles` para tratar dos arquivos do admin (junto com um servidor Web como delineado na seção anterior; isso quer dizer usar o comando de gerenciamento `collectstatic` para coletar os arquivos estáticos em `STATIC_ROOT`, e então configurar seu servidor Web para servir `STATIC_ROOT` at `STATIC_URL`), mas aqui estão três outras maneiras:

1. Crie um link simbólico para os arquivos estáticos do admin de dentro do seu “document root”(isso talvez requeira `FollowSymLinks` na sua configuração do Apache).
2. Use uma diretiva **Alias**, como demonstrado acima, para apelidar a URL apropriada (provavelmente : `settings:SATTIC_URL`+ ``admin/`)` para o local real dos arquivos do admin.
3. Copie os arquivos estáticos do admin para que entã eles fiquem dentro do “document root” do Apache.

Autenticação do banco de dados de usuário do Django vindo do Apache.

O Django disponibiliza um facilitador para habilitar o Apache a autenticar usuários diretamente no backend de autenticação do Django. veja `documentação de autenticação do mod_wsgi`.

Learn More

- About Django
- Getting Started with Django
- Team Organization
- Django Software Foundation
- Code of Conduct
- Diversity Statement

Get Involved

- Join a Group
- Contribute to Django
- Submit a Bug
- Report a Security Issue

Follow Us

[GitHub](#)

[Twitter](#)

[News RSS](#)

[Django Users Mailing List](#)

© 2005-2019 [Django Software Foundation](#) and individual contributors. Django is a [registered trademark](#) of the Django Software Foundation.

Idioma: **pt-br**

Versão da Documentação **2.2**